实验报告 Lab1

一、check_ops.py 运行截图

```
circlewwwww@LAPTOP-GEBLFOO4:~/Lab1$ make clean
make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
circlewwwww@LAPTOP-GEBLFOO4:~/Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 16/16 operators
roundEvenPow2: 14/24 operators
midpointTowardFirst: 18/32 operators
isBetweenEitherOrder: 39/48 operators
mul5Sat: 20/30 operators
classifyAdd3: 19/52 operators
floatScaleThreeHalves: 51/60 operators
floatRoundEven: 24/65 operators
float_i2f: 24/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

二、btest 运行截图

```
circlewwwww@LAPTOP-GEBLFOO4:~/Lab1$ ./btest
 Score   Rating  Errors  Function
  1       1       0       signMask
  2       2       0       bitXor
  3       3       0       negativePart
  4       4       0       copyByteWithin
  4       4       0       logicalShift
  4       4       0       swapNibblePairs
  4       4       0       secondLowestZeroBit
  5       5       0       oddParity
  5       5       0       rotateRightBits
  5       5       0       roundEvenPow2
  5       5       0       midpointTowardFirst
  7       7       0       isBetweenEitherOrder
  7       7       0       mul5Sat
  7       7       0       classifyAdd3
  7       7       0       floatScaleThreeHalves
  10      10      0       floatRoundEven
  10      10      0       float_i2f
  10      10      0       bitCount
  10      10      0       bitReverse
 Total points: 110/110
```

三、实现思路

1. 把 1 左移 31 位，得到只有最高位为 1 的掩码 0x80000000。

2. 利用恒等式 x^y = (x|y) & ~(x&y)，再用德摩根律把 x|y 改写成 ~(~x & ~y)，全程只用 ~ 和 &。

3. x >> 31 在负数时为全 1、非负时为全 0，可当作条件掩码；~x + 1 就是 -x。两者按位与：负数保留 -x，非负清零。

4. 先把 x 右移 src*8 位并取低 8 位，得到源字节；再构造掩码把 dst 字节清零；最后把源字节左移到 dst 位置，与原值按位或合并。

5. 算术右移后高位会补 1，用掩码 ~(1 << 31 >> n << 1) 把高 n 位清零，再与右移结果按位与，得到逻辑右移。

6. 用掩码 0x0F0F0F0F 取出所有低 4 位并左移 4 位，用高 4 位掩码取出高 4 位并右移 4 位，两者按位或。掩码通过 0x0F 反复左移 8、16 位拼接而成。

7. x + 1 把最低的 0 位变 1，x | (x + 1) 把最低 0 位也置 1。此时再找最低的 0 位就是原来的第二个 0 位，用 ~y & (y + 1) 得到掩码。

8. x + 1 把最低的 0 位变 1，x | (x + 1) 把最低 0 位也置 1。此时再找最低的 0 位就是原来的第二个 0 位，用 ~y & (y + 1) 得到掩码。

9. 先用 n & 31 取模，保证移位量在 0~31 之间。循环右移拆成两部分：x 左移 32-n 位保留低位到高位，x 右移 n 位保留高位，用掩码清掉多余位后按位或。

10. 构造低 n 位掩码和半程值 half = 2^(n-1)。加上 half + ((x>>n)&1) - 1，实现“大于半进位、等于半看商奇偶”的四舍六入五成双规则，最后清除低 n 位。

11. 先用 $(x\&y)+((x^y)>>1)$ 无溢出地求向下取整的平均值。当 x+y 为奇数时，若 x > y 则 floor 偏向 y，需要加 1。x > y 通过符号位分情况判断：异号时看 y 的符号，同号时看 y - x 的符号。

12. 直接用差值符号判断 x 与 a、b 的大小关系，避免溢出。分别计算 a <= x、x <= a、b <= x、x <= b 四个条件的掩码，再用 $a <= x \&\& x <= b \;||\; b <= x \&\& x <= a$ 合并，最后取符号位得到 0 或 1。

13. x*5 写成 (x<<2) + x。通过比较 x、x<<1、x<<2、x*5 的符号位是否一致来判断溢出。若溢出，根据 x 的符号返回 INT_MAX 或 INT_MIN；否则返回乘积

14. 分两步加法 s1 = x+y、s2 = s1+z，每步用 $((a^s)(b^s)) >> 31$ 检测溢出。若两步溢出方向一致（异或为 0）则不溢出；否则根据溢出的那一步的符号判断最终结果是大于 INT_MAX 还是小于 INT_MIN，返回 1 或 -1。

15. 拆出符号、阶码、尾数。NaN/Inf 原样返回。非规格化数补上隐藏位 1。尾数乘以 1.5 用 m + (m<<1) 实现，根据结果是否溢出 24 位决定右移 1 位还是 2 位，同时阶码相应调整。最后处理阶码溢出为 Inf 的情况，并保留符号。

16. 拆出符号、阶码、尾数。NaN/Inf 原样返回。根据实际指数 E 分情况：E < 0 时绝对值小于 1，E == -1 且尾数非零时舍入到 ±1，否则舍入到 ±0；E >= 23 时已是整数直接返回；否则提取整数部分和小数部分，按“大于半进位、等于半且整数部分为奇数进位”处理。舍入后重新计算阶码和尾数，舍入到零时保

留原符号。

17. 特判 0。取符号和绝对值，从最高位找到最高有效位 e，阶码为 e + 127。若 e <= 23，尾数左移补齐；否则右移丢弃多余位，并按“大于半进位、等于半且尾数最低位为 1 进位”处理。进位导致尾数溢出时，尾数清零、阶码加 1。

18. 分治法统计 1 的个数。依次用掩码 0x55555555、0x33333333、0x0F0F0F0F、0x00FF00FF、0x0000FFFF，把相邻位的计数逐层合并：先统计每 2 位，再每 4 位、每 8 位、每 16 位，最终得到 32 位的总数。大掩码通过 0x55、0x33 等小常量移位拼接。

19. 分治法反转位。先用 0xFF 和移位构造出各级掩码 m1、m2、m4、m8、m16。然后依次交换相邻 1 位、2 位、4 位、8 位，最后交换高低 16 位。每步用掩码提取需要交换的位，移位后按位或合并。